Proyecto parcial de Programación Orientada a Objetos

Fuga de la Prisión

Diseño e implementación de un videojuego 2D

Universidad Peruana Cayetano Heredia
Ingeniería Informática

Jose Fernando Chavez Lozano
Mickele Carlos Torres Velazco
Piero Cemei Romero Meza
Richard Aaron Soriano Cordova

Ciclo IV. Período académico 2026-II

I. Resumen ejecutivo

Videojuego 2D de escape y sigilo con cuatro niveles.

Su objetivo es demostrar POO mediante un juego funcional.

La arquitectura separa reglas, personajes e interfaz.

Aplica asociación, agregación y composición,

además de encapsulamiento, herencia y polimorfismo.

Implementa los patrones State y Observer.

II. Introducción

Contexto y objetivo

Proyecto del curso de POO.
Demostrar sus fundamentos y patrones en un juego de escape.

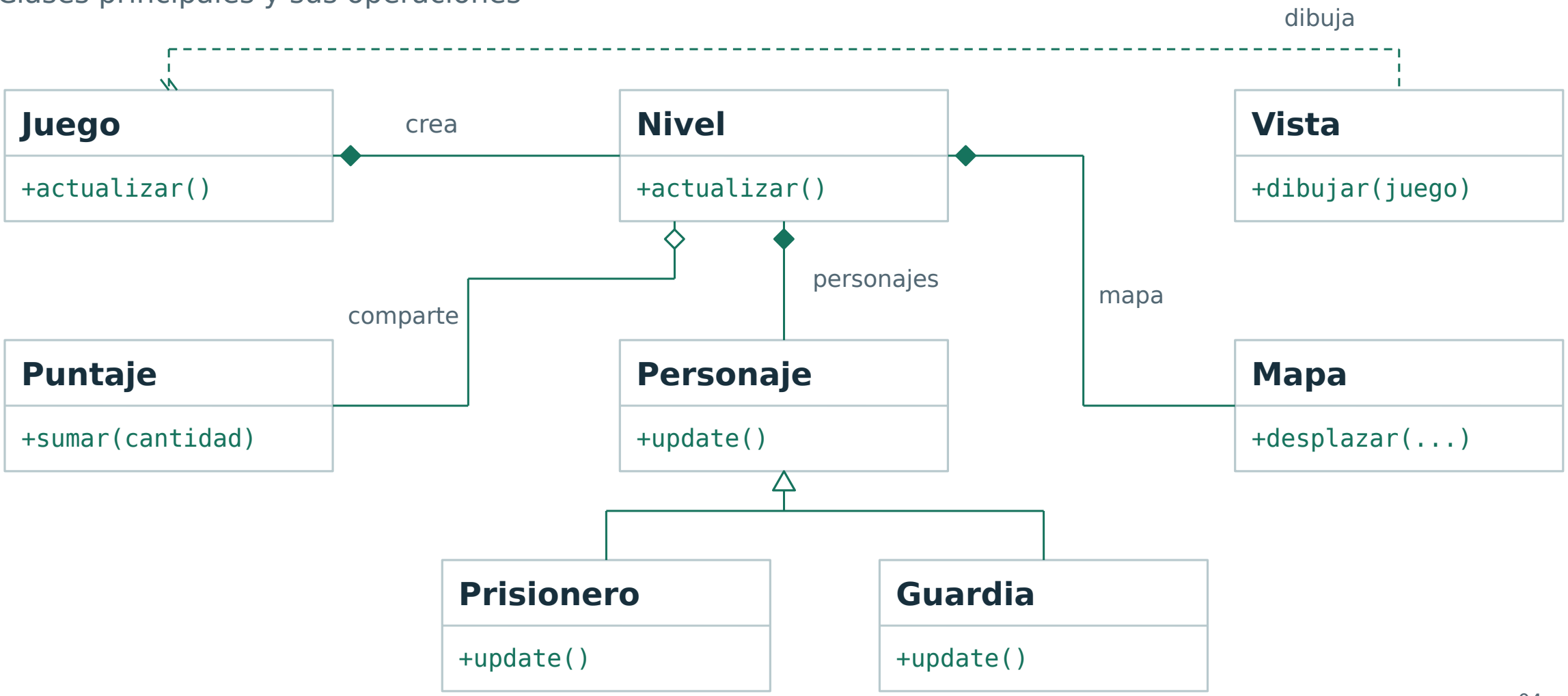
Tecnologías utilizadas

Python y Pygame: juego e interfaz.
Mermaid: diagrama UML.
Git y GitHub: control de versiones.



III. Diseño de arquitectura

Clases principales y sus operaciones



IV. Fundamentos de POO

Asociación

Personaje guarda su Nivel.

```
self.nivel = nivel
```

Agregación

Nivel recibe el Puntaje compartido de la campaña.

```
self.puntaje = puntaje
```

Composición

Nivel crea su propio Mapa.

```
self.mapa = Mapa(
    PLANOS[indice])
```

IV. Fundamentos de POO, continuación

Herencia y abstracción

Prisionero y Guardia heredan de la clase abstracta Personaje.

Polimorfismo

```
for personaje in self.personajes:
    personaje.update()
```

Encapsulamiento

Puntaje._valor se consulta con get_valor().
set_valor() valida el dato y notifica el cambio.

VI. Resultados parciales



Interfaces y funcionalidades

Apertura de celda, vigilancia, llaves y salida.
Campaña de cuatro niveles.

Uso de Git

cfaba6e Plan inicial
8917f20 Prototipo y pruebas
f3cd9fe Informe y diagramas

Ver historial del repositorio

VII. Conclusiones

Logro	Juego funcional con clases, relaciones y patrones de diseño.
Aprendizaje	Separar responsabilidades facilita probar cada regla.
Dificultad	Mantener coherentes paredes, puertas, ruido y caminos.

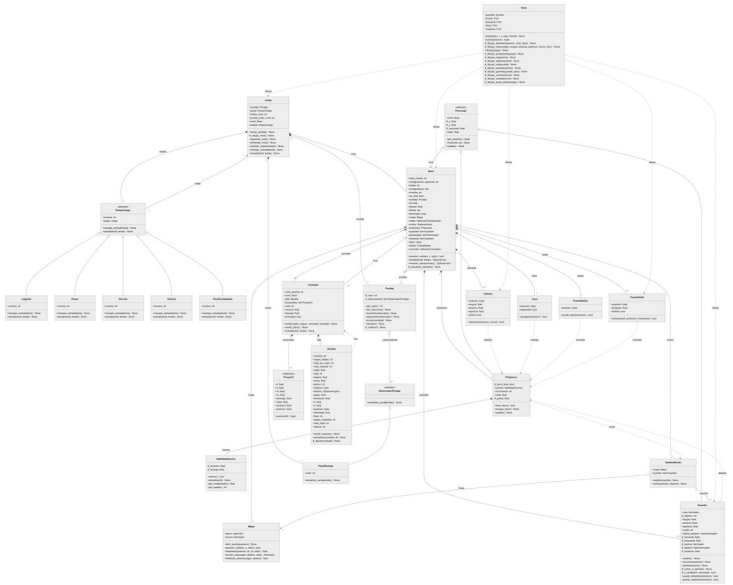
VIII. Anexos

Fragmento de código: Puntaje

```
def set_valor(self, valor):
    if type(valor) is not int or valor < 0:
        raise ValueError(
            "El puntaje debe ser un entero no negativo.")
    self._valor = valor
    self._notificar()
```

[Abrir código de Puntaje]

Diagrama de clases



[Abrir UML completo]